

项目概述

这段Go语言代码实现了均值漂移聚类算法的可视化程序，使用ebiten游戏引擎展示聚类过程。主要功能包括：

- 生成具有自然聚类特性的二维数据点
- 使用均值漂移算法自动发现数据中的聚类中心
- 可视化展示每个点向聚类中心的漂移过程
- 使用动画效果平滑过渡漂移轨迹
- 自动确定聚类数量，无需预先指定
- 显示当前迭代次数和收敛状态

该项目非常适合理解均值漂移聚类算法的工作原理，通过可视化可以直观地看到算法如何自动发现数据中的密度中心，并将数据点分配到不同的聚类中。与K-means不同，均值漂移不需要预先指定聚类数量，而是根据数据密度自动确定。

代码结构

主要组成部分

数据结构

定义了Point结构表示数据点，包含X和Y坐标

数据生成

生成具有自然聚类特性的数据点，便于展示均值漂移效果

均值漂移核心

实现了均值漂移算法的核心逻辑：计算漂移方向和更新位置

动画控制

控制聚类漂移的动画过程和状态更新

可视化

将原始数据点、漂移轨迹和最终聚类结果绘制到屏幕上

游戏循环

Ebiten框架的Update和Draw方法，控制整体流程

程序执行流程

1. 初始化：生成带聚类特性的数据点，创建均值漂移实例
2. 游戏循环开始：

- Update(): 更新动画进度, 执行均值漂移步骤
 - Draw(): 绘制当前状态的所有元素, 包括数据点、漂移轨迹和当前模式点
3. 收敛判断: 当所有点的移动距离小于阈值时, 算法收敛
 4. 聚类标签分配: 根据最终的模式点位置, 将相似的模式点合并为同一聚类

核心算法

均值漂移算法

均值漂移是一种基于密度估计的非参数聚类方法, 它的核心思想是:

每个数据点都会向数据密度更高的区域"漂移", 最终收敛到密度峰值 (模式点)

算法原理

给定一个点 x , 均值漂移迭代更新公式为:

$$x_{i+1} = \sum_{j=1}^n K(x_i - x_j) \cdot x_j / \sum_{j=1}^n K(x_i - x_j)$$

其中:

- K 是核函数 (通常使用高斯核)
- n 是数据点数量
- x_j 是第 j 个数据点

代码实现

核心算法在Step()方法中实现:

```
func (ms *MeanShift) Step() bool {
    converged := true
    bandwidthSq := ms.bandwidth * ms.bandwidth // 带宽平方 (优化计算)

    // 对每个点执行一次漂移计算
    for i := range ms.modes {
        currentMode := ms.modes[i]
        sumX, sumY := 0.0, 0.0
        totalWeight := 0.0

        // 计算带宽范围内的加权平均
        for _, p := range ms.points {
            distSq := (p.X-currentMode.X)*(p.X-currentMode.X) + (p.Y-currentMode.Y)*(p.Y-currentMode.Y)
            if distSq <= bandwidthSq {
                // 高斯核函数权重
                weight := math.Exp(-distSq / (2 * bandwidthSq))
                sumX += p.X * weight
                sumY += p.Y * weight
                totalWeight += weight
            }
        }

        // 计算新的模式点
        if totalWeight > 0 {
            newMode := Point{
                X: sumX / totalWeight,
                Y: sumY / totalWeight,
            }
            ms.modes[i] = newMode
        }

        // 检查是否收敛 (移动距离小于阈值)
```

```
        if distance(newMode, currentMode) > 0.01 {
            ms.modes[i] = newMode
            converged = false
        }
    }

    ms.iterations++
    return converged
}
```

高斯核函数

在代码中使用的高斯核函数：

```
weight := math.Exp(-distSq / (2 * bandwidthSq))
```

高斯核函数的特点：

- 距离越近的点权重越大
- 带宽参数控制了影响范围
- 随着距离增加，权重呈指数衰减

算法特性

均值漂移算法的主要优点：

- 不需要预先指定聚类数量
- 能够发现任意形状的聚类
- 对噪声和离群点具有鲁棒性
- 算法自动确定聚类中心的位置

主要缺点：

- 计算复杂度较高 ($O(n^2)$)
- 带宽参数需要手动调整
- 可能会收敛到局部密度峰值

可视化部分

绘制元素

原始数据点

使用浅灰色小点表示：

- 每个点大小为2x2像素
- 作为背景显示，不随算法迭代变化
- 提供参考框架，帮助理解漂移过程

漂移轨迹

使用浅灰色线条连接原始点和当前模式点：

- 每条线从原始点出发，指向当前漂移模式

- 母线线从原始点出发，指向当前漂移位置
- 线的颜色为浅灰色，不干扰主要视觉元素
- 帮助理解每个点的漂移路径

当前模式点

使用彩色方块表示每个点的当前漂移位置：

- 算法收敛前使用统一蓝色
- 算法收敛后根据聚类结果使用不同颜色
- 每个点大小为4x4像素，便于观察

状态信息

显示当前算法状态：

- 当前迭代次数
- 带宽参数值
- 是否收敛的状态提示

动画控制

动画效果通过Update()和UpdateAnimation()方法控制：

```
func (g *Game) Update() error {
    if g.ms.converged {
        return nil
    }

    // 动画进度更新
    g.animProg += g.animSpeed
    if g.animProg >= 1.0 {
        // 动画结束，执行下一步均值漂移
        g.animProg = 0
        g.ms.converged = g.ms.Step()
        g.ms.currentStep++
    }

    // 更新当前动画帧的显示状态
    g.ms.UpdateAnimation(g.animProg)
    return nil
}
```

关键动画逻辑：

- 使用animProg变量跟踪动画进度（0-1）
- 每次更新增加animSpeed（0.03）
- 动画完成后执行均值漂移迭代步骤
- 在UpdateAnimation()中使用线性插值计算当前显示位置

坐标映射

将数学坐标（0-100范围）映射到屏幕坐标：

```
// 绘制原始数据点（灰色小点点）
for _, p := range g.ms.points {
    x := int(p.X * float64(g.width) / 100)
    y := int(p.Y * float64(g.height) / 100)
```

```
for dx := -1; dx <= 1; dx++ {
    for dy := -1; dy <= 1; dy++ {
        screen.Set(x+dx, y+dy, color.Gray{Y: 200})
    }
}
```

这种映射确保生成的随机点（0-100范围）能够正确显示在窗口内。

参数说明

关键参数

参数	含义	默认值	调整建议
bandwidth	高斯核函数带宽	5.0	控制聚类粒度，较大值产生更少的聚类，较小值产生更多的聚类
width, height	窗口尺寸	800x600	根据需要调整窗口大小
animSpeed	动画速度	0.03	值越大动画越快，可调整为0.01-0.05之间
total	数据点总数	600	数据越多，聚类效果越明显，但计算量也越大
clusters	生成的自然聚类数量	5	控制初始数据的聚类特性，用于测试算法效果

初始化参数

在main()函数中可以调整的参数：

```
func main() {
    // 生成600个带聚类特性的点（5个自然聚类）
    points := generateClusteredPoints(600, 5)

    // 初始化均值漂移（带宽设为5.0，控制聚类粒度）
    game := NewGame(points, 5)
    ebiten.SetWindowSize(game.width, game.height)
    ebiten.SetWindowTitle("均值漂移聚类动画")

    // 运行动画
    if err := ebiten.RunGame(game); err != nil {
        panic(err)
    }
}
```

通过修改这些参数，可以观察不同条件下均值漂移算法的表现。特别注意，bandwidth参数对聚类结果有重要影响：

- 较大的bandwidth会导致更少、更大的聚类
- 较小的bandwidth会导致更多、更小的聚类
- 合适的bandwidth值取决于数据分布的特性